Tecnologia-y-Arquitectura-Robotica / practica_2_2526

Code Issues Pull requests Agents Actions Projects Security Insights

Watch 0

0 stars 0 forks 0 watching Branches Activity Custom properties
Tags

Private repository

main 1 Branch 0 Tags Go to file t Go to file Code

TamaiRamirez Index 8cb41ec · 2 weeks ago

Parte_1	Actualizado Guion	2 weeks ago
Parte_2	Init Practica 2	2 weeks ago
images	Actualizado Guion	2 weeks ago
README.md	Index	2 weeks ago

README

Práctica 2: Trabajando con el Turtlebot 3 en simulación

Índice

1. Entrega
2. Instalación Turtlebot 3
3. Parte 1: Primeros Pasos con el Turtlebot y Odometría
   i. Ejercicios
4. Parte 2: Resolución de un Laberinto
   i. Ejercicios

Entrega

La entrega de esta práctica se realizará a través de la herramienta de 'Evaluación' de UaCloud. La misma debe ser una memoria en formato `.pdf` en la cual se encuentren las respuestas a las preguntas teóricas y a los ejercicios propuestos de las diferentes partes. Deberá tener el nombre de *Apellidos_Nombre.pdf*. Asimismo, esta práctica **se debe hacer en parejas o grupos de tres** (el nombre del `.pdf` debe ser el primer apellido e inicial de los integrantes y en la memoria también se deben añadir, Ej: `Ramirez_T_Pujol_F.pdf`). En esta práctica se deben entregar también los códigos que se hayan generado para resolver los ejercicios. Pueden o bien compartir un enlace al repositorio de `GitHub` que estén usando, o añadirnos como colaboradores (Nuestros usuarios de github son `TamaiRamirezUA` y `bigpacopujol`, aunque nos añadan como colaboradores, añadan el enlace al repositorio en la memoria) o compartartan enlace de `Drive`, como prefieran. Por otro lado, es **imprescindible** que graben la resolución de los ejercicios y los compartan en la memoria a través de un enlace también, así se puede observar la correcta ejecución de los ejercicios.

Instalación Turtlebot 3

Para comenzar, debemos lanzar nuestro contenedor de ROS Humble, ejecutanto `run.sh`, si no lo hemos hecho anteriormente. Una vez dentro debemos ejecutar los siguientes comandos en la terminal:

```
cd ros2_ws/src/

# Clonamos el repositorio que contiene los paquetes del Turtlebot 3
git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3.git
git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_msgs.git
git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_simulations.git

# Compilamos los paquetes
cd ..

sudo apt update
sudo apt install gazebo libgazebo-dev ros-humble-gazebo-ros-pkgs ros-humble-dynamixel-sdk

source /opt/ros/humble/setup.bash

source install/setup.bash

export GAZEBO_PLUGIN_PATH=/opt/ros/humble/lib
export GAZEBO_MODEL_PATH=/opt/ros/humble/share

colcon build --symlink-install
```

IMPORTANTE: Cada vez que entremos en el contenedor de ROS 2 ya sea ejecutando `./run.sh` o si accedemos al contenedor, una vez lanzado por el script anterior, en nuevas terminales para ejecutar diferentes herramientas de ROS ejecutando `./connect_ros.sh`, necesitamos que nuestro sistema tenga conocimiento de las variables del entorno. Para evitar tener que ejecutar siempre los mismos comandos constantemente cada vez que iniciamos el contenedor o generamos una nueva terminal, debemos ejecutar lo siguiente:

1. Configurar el archivo `~/.bashrc` para que automáticamente establezca las variables de entorno en nuestro espacio de trabajo `ros2_ws`:

```
nano ~/.bashrc
```

2. Dentro de este archivo en la última línea escribiremos lo siguiente:

```
source /opt/ros/humble/setup.bash
source /workspace/ros2_ws/install/setup.bash
export LIBGL_ALWAYS_SOFTWARE=1
export LIBGL_DEBUG=verbose
export GAZEBO_PLUGIN_PATH=/opt/ros/humble/lib
export GAZEBO_MODEL_PATH=/opt/ros/humble/share
```

3. Guardaremos el archivo actualizado. No obstante, esto no asegura que cada vez que ejecutemos de nuevo el contenedor, esta actualización se haya guardado. Esto se debe a que el contenedor se resetea cada vez que se cierra para evitar actualizarse automáticamente y que pueda que haya incompatibilidades en algún momento con algún cambio que hagamos, de esta forma, si hay algo que hemos tocado que haga una incompatibilidad, con cerrar y volver a lanzar el contenedor sería suficiente. En este caso, sí que nos interesa que el cambio se mantenga, por tanto, abriremos una nueva terminal (sin haber salido y cerrado el contenedor) y ejecutaremos el siguiente comando:

```
docker commit ros_humble ros_humble:latest
```

Por último, para que los cambios se guarden correctamente, es necesario cerrar el contenedor y volver a lanzarlo.

El paquete del `Turtlebot 3` integra tres opciones de robot: `Burger, Waffle y Waffle Pi`. En nuestro caso, vamos a emplear el robot `Waffle` y `Burger` para la realización de esta práctica. A continuación, vamos a comprobar que todo se haya instalado y compilado de forma correcta. Para ello, vamos a simular el robot dentro del simulador `Gazebo`:

Terminal 1:

```
source /opt/ros/humble/setup.bash
source /workspace/practica_2/ros2_ws/install/setup.bash
# Exportar el modelo de robot Turtlebot 3 (Esto debemos hacerlo siempre que se abra una nueva terminal)
export TURTLEBOT3_MODEL=waffle
```

```
ros2 launch gazebo_ros gazebo.launch.py
```

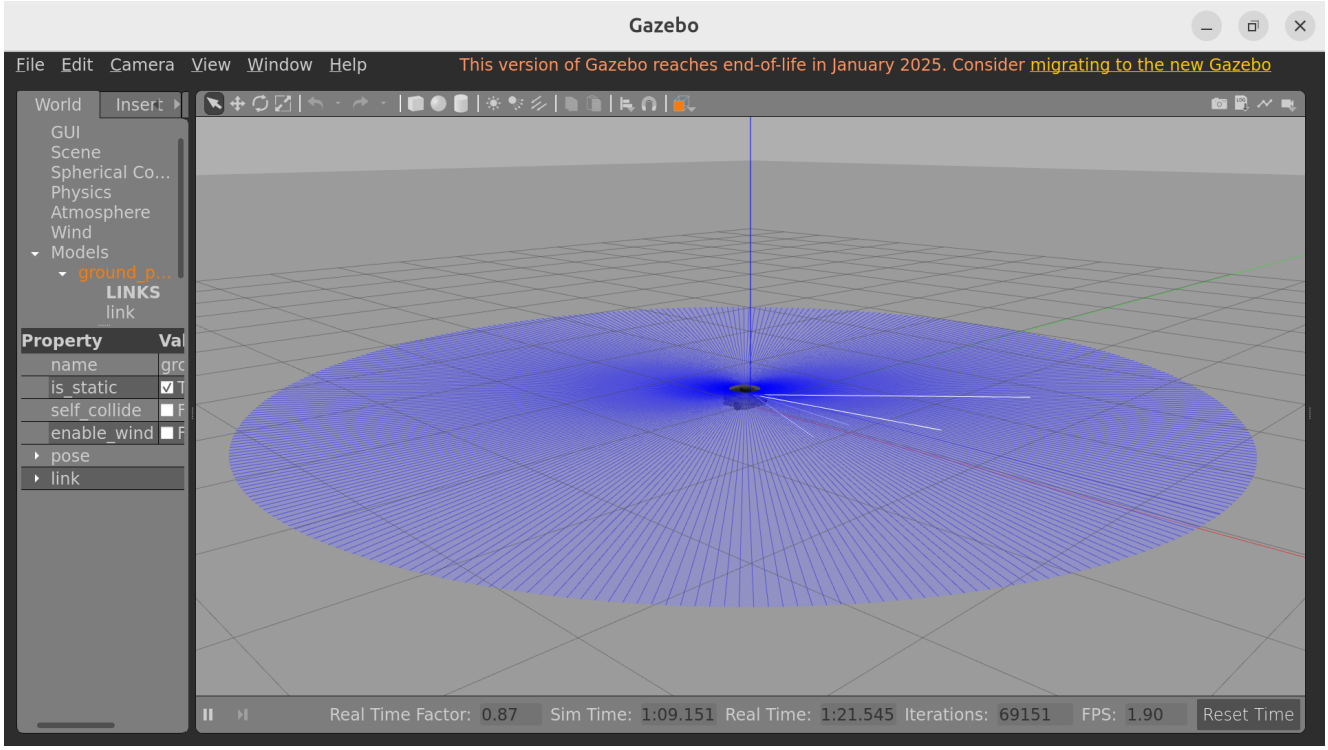
NOTA: Antes de ejecutar lo siguiente, esperen a que se cargue `Gazebo`, esto le llevará unos minutos la primera vez que se lanza.

Terminal 2:

```
source /opt/ros/humble/setup.bash
source /workspace/practica_2/ros2_ws/install/setup.bash
# Exportar el modelo de robot Turtlebot 3 (Esto debemos hacerlo siempre que se abra una nueva terminal)
export TURTLEBOT3_MODEL=waffle

ros2 launch turtlebot3_gazebo spawn_turtlebot3.launch.py
```

Al lanzar estos comandos debería abrirse `Gazebo` y mostrar lo siguiente:



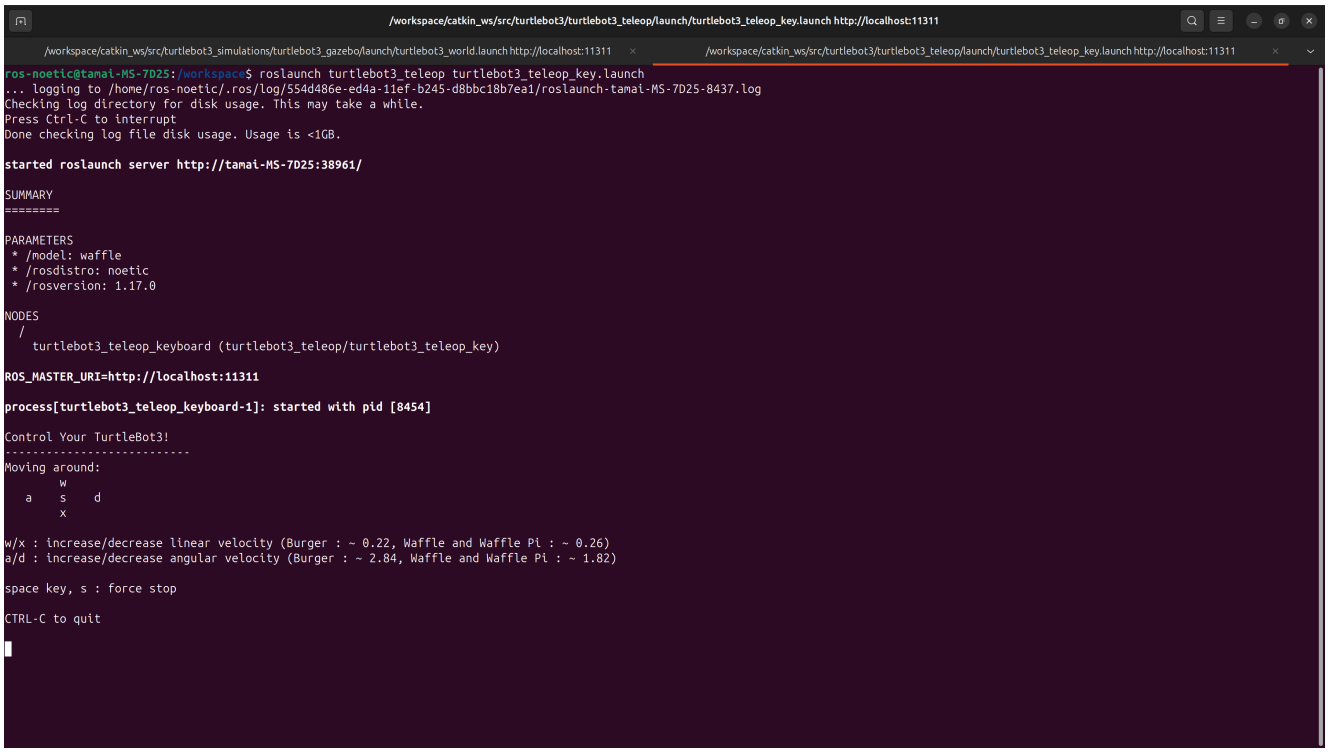
El lanzar el Turtlebot3 de esta manera es debido a que primero es necesario que se cargue `Gazebo` para luego poder incluir el robot. Esto sólo deberemos hacerlo cuando necesitemos lanzar el robot en un mundo vacío y al usar la paquetería oficial de ROS2 para este robot. Para los ejercicios propuestos en los que se les proporciona un mapa concreto, esto no será necesario ya que se les proporcionará los archivos `launch` para lanzarlo todo a la vez.

Parte 1: Primeros Pasos con el Turtlebot y Odometría

En esta primera parte vamos a aprender a manejar el Turtlebot 3, concretamente el `Waffle`. Para ello debes lanzar la simulación del robot en `Gazebo` como se ha mencionado anteriormente. Una vez lanzado, lo primero que vamos a hacer es teleoperar el robot. En una nueva terminal ejecuta:

```
export TURTLEBOT3_MODEL=waffle
ros2 run turtlebot3_teleop teleop_keyboard
```

La terminal debe mostrar lo siguiente:



Como puedes observar, ahora con el teclado podemos teleoperar al robot y moverlo por el entorno de simulación. Con la tecla `w` el robot realizará un movimiento lineal hacia adelante/atrás, con las teclas `a/d` girará hacia la izquierda/derecha y con la tecla `s` se parará el robot. **NOTA**: El Turtlebot 3 se teleopera en velocidad, por tanto, lo que se hace es darle una velocidad lineal/angular para moverlo, esto implica que si lo no paramos, se moverá indefinidamente.

> Pregunta 1: ¿Cuál es el topic en el cual se debe publicar la información para que el robot se mueva?
>
> Pregunta 2: ¿Cuál es el tipo de mensaje que se publica?
>
> Pregunta 3: ¿Qué tipo de movimientos puede realizar el robot? ¿Cuáles son los ejes de movimientos positivos (Por ejemplo, si le decimos al robot que gire -20º girará a izquierda o a derecha)?
>
> Pregunta 4: ¿El robot gira sobre su eje siempre? ¿Qué comando de odometría se le está mandando a los motores para que sea así?
>
> Pregunta 5: ¿Cuál es la unidad de magnitud de las velocidades lineales y angulares?

Ejercicios:

1. Genera un paquete de ROS con las dependencias necesarias llamado `p3_pkg`. En este paquete genera un nodo llamado `movimiento.py` que permita en base al valor entero pasado por la línea de comandos, realice los siguientes movimientos:
   - 0: Realizar un movimiento lineal haciendo que el robot avance 2 metros.

     > Pregunta 6: ¿Cómo puedo comprobar que el robot haya avanzado esta distancia? (Pista: Revisa los topics del robot)
   - 1: Realiza un movimiento que dibuje un triángulo equilátero, de forma que el robot vuelva a su posición inicial (aproximadamente). Cada lado del triángulo debe medir 3 metros.
   - 2: Realiza un movimiento que dibuje un cuadrado, de forma que el robot vuelva a su posición inicial (aproximadamente). Cada lado del cuadrado debe medir 1 metro.
   - 3: Realiza un movimiento que dibuje un "infinito", de forma que el robot vuelva a su posición inicial (aproximadamente). En este caso, el robot debe avanzar 0.5 metros y realizar un giro de 60º (π/3 radianes), a partir de aquí, el robot debe dibujar el "infinito" de forma que acabe en la misma posición que de partida (aproximadamente).

NOTA: Para no tener que cancelar la simulación cada vez que quieras resetear el robot y ubicarlo en el origen, debes tanto resetear `Gazebo` como resetear la odometría del robot. Para ello, primero desde la ventana de `Gazebo` haz `Ctrl+R`. A continuación, resetea la odometría con el siguiente comando:

```
ros2 service call /reset_simulation std_srvs/srv/Empty "{}"
```

Para comprobar cómo ha realizado el robot los movimientos, sería interesante que se "pinte" en un gráfico la posición del robot a lo largo del tiempo (usando la librería `matplotlib` que ya está instalada). En este caso, debes generar un nodo llamado `dibuja_mov.py`. Este nodo debe suscribirse al topic correspondiente y guardar las posiciones del robot para luego dibujarlas.

2. Ejecuta 10 veces el movimiento del **cuadrado, del triángulo y del infinito** del Ejercicio 1 y dibuja el movimiento que realiza el robot como en el Ejercicio 1:

> Pregunta 7: ¿El robot acaba en la posición original? ¿A qué puede deberse esto?

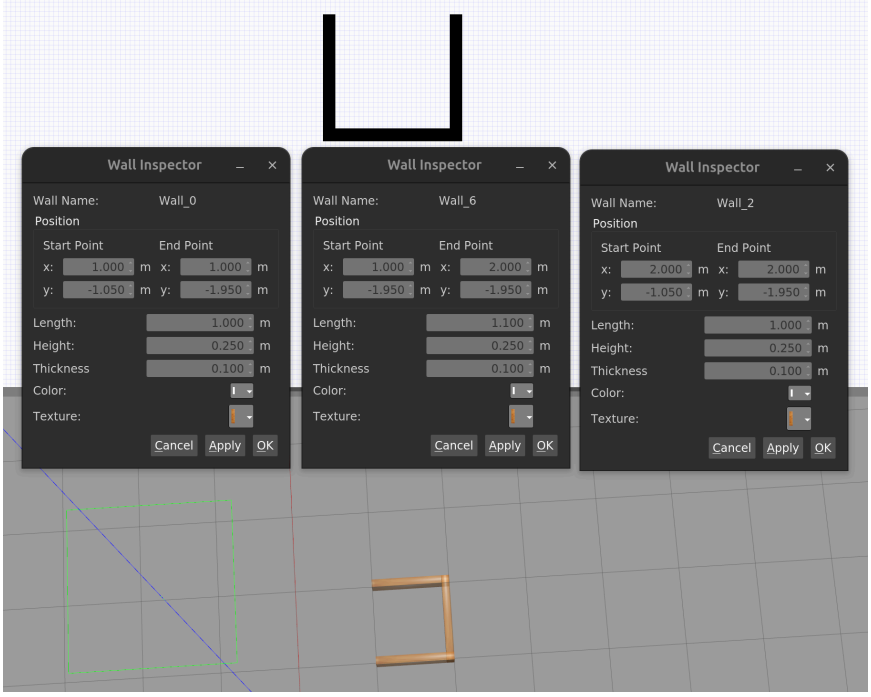
3. En este ejercicio vamos a comprobar qué tal se les da las habilidades de aparcamiento. El objetivo es emplear el Turtlebot 2 y desarrollar un nodo que ejecute el movimiento del aparcamiento y lo "aparque" dentro de un espacio concreto, el nodo debe llamarse `aparcamiento.py` dentro del paquete `p3_pkg`. Para poder cargar el entorno de simulación con el mapa para aparcar, debes añadir los directorios `models`, `worlds` y `launch`, que se encuentran en la carpeta `Parte_1`, en el directorio raíz del paquete.

Para que funcione esta parte, debes modificar el archivo `setup.py` del siguiente modo, añadir las dependencias de `ros2launch` en el `package.xml y volver a compilar el paquete:

```
import os
from glob import glob

data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        (os.path.join('share', package_name, 'launch'), glob('launch/*.py')),
        (os.path.join('share', package_name, 'worlds'), glob('worlds/*.world')),
        (os.path.join('share', package_name, 'models', 'parking'), glob('models/parking/*')),
    ],
```

A continuación, para poder lanzar el robot debes ejecutar el siguiente comando una vez hayas compilado el paquete: `bash ros2 launch p3_pkg parking_tb3.launch.py` Los datos de cómo se encuentra dispuesto el espacio donde aparcar los puedes visualizar en la siguiente imagen:



4. Como podrás observar, el robot no ejecuta las trayectorias de forma perfecta por defecto en el Ejercicio 1 y 2. Investiga cómo corregir estos errores en la odometría para que el robot ejecute los movimientos de forma correcta. Explica detalladamente cómo has logrado que el robot ejecute la trayectoria. (**PISTA**: Busca como implementar un controlador que corrija los giros del robot)

5. Repite los ejercicios anteriores, pero esta vez usando el modelo del Turtlebot 3 `Burger` y contesta a las siguientes preguntas:

> Pregunta 5: Al ejecutar los mismos movimientos que en los Ejercicios 1 y 2, Al ejecutar los mismos movimientos que en los Ejercicios 1 y 2: ¿Observas diferencias en la precisión de los movimientos entre `Burger` y `Waffle` ? ¿Cuál robot acumula más error al finalizar trayectorias como el cuadrado o el triángulo? ¿En qué tipo de movimiento se nota más la diferencia: lineal, rotacional o combinado?
>
> Pregunta 8: ¿ `Burger` realiza giros más amplios o más cerrados que Waffle?
>
> Pregunta 9: Al implementar controladores para corregir errores de trayectoria: ¿Los mismos parámetros funcionan igual para ambos robots? ¿Cuál robot requiere ajustes más finos de los parámetros del controlador para mantener la trayectoria correcta? ¿Alguno se vuelve inestable con los mismos parámetros?
>
> Pregunta 10: Durante la tarea de aparcamiento: ¿Cuál robot realiza la maniobra con menos correcciones? ¿Alguno choca o queda demasiado cerca de los obstáculos? ¿Notas diferencias en la respuesta a comandos de giro o avance rápido?
>
> Pregunta 11: Inspecciona las propiedades URDF de ambos robots (wheel radius, wheel separation, masa, sensores): ¿Cómo afectan estas diferencias a la precisión de los movimientos, la estabilidad y la odometría? (**Pista**: Esta información se encuentra dentro del paquete `turtlebot3_simulations/turtlebot3_gazebo/urdf`)

En este caso para realizar el aparcamiento, los comandos para lanzar el entorno de simulación con el Turtlebot 3 son:

```
export TURTLEBOT3_MODEL=burger
ros2 launch p3_pkg parking_tb3.launch.py
```

Parte 2: Resolución de un Laberinto

En esta segunda parte de la práctica, se utilizará el Turtlebot 3 equipado con un sensor LiDAR para resolver un laberinto de manera autónoma. Para ello, deberán crear un nuevo paquete de ROS llamado `maze_pkg` , asegurándose de incluir todas las dependencias necesarias. Este paquete contendrá los nodos y scripts requeridos para la navegación del robot dentro del laberinto, empleando técnicas de percepción y control basadas en los datos obtenidos del LiDAR. Para comenzar, lanza en un mundo vacío por un lado el Turtlebot 2 y por otro el Turtlebot 3 y contesta a las siguientes preguntas para cada robot:

> Pregunta 12: ¿Cuál es el topic asociado al LiDAR? ¿Cuál es la tipología de los mensajes?
>
> Pregunta 13: ¿Cuál es el rango de **distancias** que puede medir el LiDAR? ¿Cuál es el rango **angular de escaneo** que tiene el LiDAR? ¿Cuál es el **origen de referencia** del LiDAR?

Ejercicios:

1. El objetivo es emplear el Turtlebot 3 y desarrollar un nodo que haga que el robot logre "escapar" del laberinto, el nodo debe llamarse `res_maze.py` dentro del paquete `maze_pkg`. Para poder cargar el entorno de simulación con el mapa con el laberinto, debes añadir los directorios `models`, `worlds` y `launch`, que se encuentran en la carpeta `Parte_2`, en el directorio raíz del paquete. A continuación, para poder lanzar el robot debes ejecutar el siguiente comando una vez hayas compilado el paquete:

Releases

No releases published

Packages

No packages published

Contributors

No contributors

Languages

- **Python** 100.0%